07 ARRAYS AND STRINGS • THEORY

03 c strings vs std string

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

03 — C-Style Strings (`char[]`) vs. Modern `std::string`

1. C-Style Null-Terminated Character Arrays

A C-string is a `char` array terminated by the null character `'\0'` (ASCII `0`):

```cpp
char name[6] = "BTech"; // Stores {'B', 'T', 'e', 'c', 'h', '\0'} — requires 6 bytes for 5 letters!
```

If you write `char bad[5] = {'B','T','e','c','h'};` without `'\0'`, passing `bad` to `cout << bad` or `strlen(bad)` reads past the end of the array and causes **Undefined Behavior**.

2. Modern `std::string` (`#include <string>`)

```cpp
string course = "Mathematics";
course += " and Computing";
cout << course << " (Length: " << course.length() << ")" << endl;
```

Why `std::string` is Preferred Over `char[]`

- Automatically resizes to prevent buffer overflows.
- Supports direct assignment (`s1 = s2`), concatenation (`s1 + s2`), and comparison (`s1 == s2`), whereas comparing two `char[]` arrays with `s1 == s2` compares their **memory addresses** instead of their text!